

Quiz Mi-Parcours — Sessions 1 à 10

Informations

Déroulement

- Durée : **3 heures**.
- Ce quiz couvre les sessions 1 à 10 : géométrie vectorielle, cinématique, lois de Newton, forces, intégration d'Euler, impulsion et collisions, broad/narrow phase, Verlet + PBD, moteur de particules, et rotations.
- Le quiz est **interactif** : on le traverse ensemble, question par question, avec discussion.
- Format : choix multiples (QCM). Pour chaque question, la bonne réponse **est** l'explication.
- Documents autorisés : vos notes de cours et le PDF du cours. Pas d'internet.

Partie A – Géométrie et Vecteurs (Session 1)

A1. Produit scalaire – QCM

Soit $\vec{u} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$ et $\vec{v} = \begin{pmatrix} -4 \\ 3 \end{pmatrix}$. Que vaut $\vec{u} \cdot \vec{v}$, et que dit ce résultat sur l'angle entre $\vec{u}$ et $\vec{v}$?

- 0 – le produit scalaire est nul, ce qui signifie que les vecteurs sont **perpendiculaires** (angle de 90°). C'est le test le plus rapide pour vérifier une orthogonalité.
- 25 – c'est $\|\vec{u}\| \cdot \|\vec{v}\| = 5 \times 5$, mais le produit scalaire réel est $3 \times (-4) + 4 \times 3 = 0$. On a confondu norme et produit scalaire.
- 25 – c'est l'opposé du produit des normes, pas le produit scalaire.
- 7 – c'est $3 + 4$, la somme des composantes de $\vec{u}$, pas le produit scalaire.

A2. Vecteur unitaire – QCM

Soit $\vec{v} = \begin{pmatrix} 6 \\ 8 \\ 0 \end{pmatrix}$. Le vecteur unitaire $\hat{u} = \frac{\vec{v}}{\|\vec{v}\|}$ dans la direction de $\vec{v}$ vaut :

- $\begin{pmatrix} 0.6 \\ 0.8 \\ 0 \end{pmatrix}$ – on divise chaque composante par la norme $\|\vec{v}\| = 10$. Le résultat est de longueur 1.

Un vecteur unitaire représente une **direction pure**, sans magnitude.

- $\begin{pmatrix} 6 \\ 8 \\ 0 \end{pmatrix}$ – c'est le vecteur original, pas normalisé ($\|\vec{v}\| = 10$, pas 1).
- $\begin{pmatrix} 3 \\ 4 \\ 0 \end{pmatrix}$ – on a divisé par 2, mais ce n'est pas unitaire ($\|\vec{u}\| = 5$).
- $\begin{pmatrix} 0.8 \\ 0.6 \\ 0 \end{pmatrix}$ – les composantes sont inversées ; ce n'est plus dans la direction de $\vec{v}$.

A3. Produit vectoriel – QCM

Vous programmez un personnage qui marche sur un terrain 3D. Vous connaissez deux vecteurs non parallèles qui définissent une face du terrain. Vous avez besoin de la **normale** de cette face (un vecteur perpendiculaire à la surface). Quelle opération utilisez-vous ?

- Le **produit vectoriel** ($\vec{a} \times \vec{b}$) – il produit toujours un vecteur **perpendiculaire** à ses deux entrées. C'est l'outil standard pour calculer une normale de surface.
- Le **produit scalaire** ($\vec{a} \cdot \vec{b}$) – il produit un **scalaire**, pas un vecteur. On ne peut pas obtenir une normale avec.
- La **soustraction** ($\vec{a} - \vec{b}$) – elle donne un vecteur dans le **plan** des deux entrées, pas perpendiculaire.
- La **normalisation** ($\hat{u} = \frac{\vec{v}}{\|\vec{v}\|}$) – elle donne un vecteur **unitaire** mais dans la **même direction** que l'entrée, pas perpendiculaire.

A4. Produit scalaire – QCM

Lors d'une collision, vous avez la vitesse $\vec{v}$ d'un objet et la normale $\hat{n}$ de la surface. Vous devez **décomposer** la vitesse en composante **normale** (vers la surface) et **tangentielle** (le long de la surface). Quelle opération vous donne la composante normale scalaire ?

1. Le **produit scalaire** $(\vec{v} \cdot \hat{n})$ — il projette $\vec{v}$ sur $\hat{n}$ et donne un **scalaire** : la magnitude de la composante normale. C'est exactement ce qu'on utilise dans le calcul d'impulsion ($v_{\text{rel}} = \vec{v} \cdot \hat{n}$).
2. Le **produit vectoriel** $(\vec{v} \times \hat{n})$ — il donne un vecteur **perpendiculaire** aux deux, pas une composante projetée.
3. La **soustraction** $(\vec{v} - \hat{n})$ — elle n'a pas de sens physique ici ; on ne soustrait pas un vecteur unitaire à une vitesse.
4. La **normalisation** $(\frac{\vec{v}}{\|\vec{v}\|})$ — elle donne la **direction** de la vitesse, pas sa composante sur la normale.

Partie B — Cinématique 2D (Session 2)

B1. Projectile — QCM

Dans un mouvement de projectile **sans résistance de l'air**, que peut-on dire de la composante horizontale de la vitesse v_x ?

1. Elle reste **constante** — il n'y a aucune force horizontale, donc aucune accélération horizontale. La gravité n'affecte que la composante verticale v_y .
2. Elle diminue linéairement avec le temps — la gravité est verticale, elle n'affecte pas v_x .
3. Elle augmente avec l'accélération gravitationnelle — la gravité est verticale, pas horizontale.
4. Elle devient nulle au sommet de la trajectoire — au sommet, c'est v_y qui est nulle, pas v_x .

B2. Vitesse comme dérivée — QCM

La vitesse est la **dérivée** de la position. Un objet a une position $\vec{r}(t) = \begin{pmatrix} 2t \\ t^2 \end{pmatrix}$. Que vaut sa vitesse $\vec{v}(t)$?

1. $\vec{v}(t) = \begin{pmatrix} 2 \\ 2t \end{pmatrix}$ — on dérive chaque composante : $\frac{d}{dt}(2t) = 2$, $\frac{d}{dt}(t^2) = 2t$. La vitesse est le **taux de variation** de la position.
2. $\vec{v}(t) = \begin{pmatrix} 2t \\ t^2 \end{pmatrix}$ — c'est la position, pas la vitesse. On n'a pas dérivé.
3. $\vec{v}(t) = \begin{pmatrix} 2 \\ t \end{pmatrix}$ — on a dérivé t^2 en t au lieu de $2t$ (oubli du facteur 2).
4. $\vec{v}(t) = \begin{pmatrix} 2t \\ 2t \end{pmatrix}$ — on a dérivé le mauvais terme ($2t$ au lieu de le garder).

B3. Mouvement circulaire — QCM

Un personnage tourne sur un manège à vitesse **constante**. Que peut-on dire de son accélération ?

1. Elle pointe **vers le centre** du cercle — c'est l'accélération **centripète**. Même si la vitesse scalaire est constante, la **direction** change, donc il y a une accélération. Sans elle, le personnage irait en ligne droite (1ère loi de Newton).
2. Elle est **nulle** — la vitesse étant constante, il n'y a pas d'accélération. Faux : la **direction** change, pas seulement la magnitude.
3. Elle pointe **vers l'extérieur** (centrifuge) — c'est une force **ressentie** dans le référentiel tournant, pas l'accélération réelle.
4. Elle est **tangente** au cercle — ce serait le cas si la vitesse augmentait, pas si elle est constante.

Partie C — Lois de Newton (Session 3)

C1. Deuxième loi — QCM

La deuxième loi de Newton dit $\vec{F} = m \cdot \vec{a}$. Si on **double la masse** d'un objet tout en gardant la **même force** appliquée, que devient son accélération ?

1. Elle est **divisée par 2** — car $a = \frac{F}{m}$, donc si m double, a diminue de moitié. Plus de masse = plus d'inertie = moins d'accélération pour la même force.
2. Elle est **doublée** — la masse donne plus d'inertie, donc **moins** d'accélération, pas plus.
3. Elle **ne change pas** — l'accélération dépend de la force **et** de la masse ($a = \frac{F}{m}$).
4. Elle est **multipliée par 4** — il n'y a aucune raison que la masse et la force se multiplient.

C2. Troisième loi — QCM

Une balle de 0.5 kg entre en collision avec un mur fixe. La balle exerce une force de 100 N sur le mur. Quelle force le mur exerce-t-il sur la balle ?

1. 100 N dans le sens opposé (vers la balle) — la troisième loi dit que l'action et la réaction sont **égales et opposées**, indépendamment des masses. Le mur ne bouge pas parce qu'il est ancré, mais il exerce quand même 100 N.
2. 0 N — le mur est fixe, il ne bouge pas. Faux : ne pas bouger ne signifie pas ne pas exercer de force.
3. 100 N dans le même sens que la balle — la réaction est **opposée** à l'action, pas dans le même sens.
4. 50 N — la masse de la balle n'entre **pas** en compte dans la troisième loi. La réaction est toujours égale à l'action.

C3. Chute libre — QCM

Pourquoi deux objets de masses différentes (1 kg et 100 kg) tombent-ils avec la même accélération dans le vide ?

1. Parce que la masse s'annule : $a = \frac{F}{m} = \frac{mg}{m} = g$ — la force gravitationnelle est **proportionnelle** à la masse, donc quand on divise par la masse pour obtenir l'accélération, elle s'annule. L'accélération est g , indépendante de m .
2. Parce que la gravité ne dépend pas de la masse — faux, la force de gravité **dépend** de la masse ($F = mg$). C'est l'accélération qui n'en dépend pas.
3. Parce que les objets ont le même poids — faux, le 100 kg est 100 fois plus lourd. Mais il est aussi 100 fois plus difficile à accélérer.
4. Parce qu'il n'y a pas d'air — l'absence d'air supprime le **drag**, mais la raison de l'égalité est l'annulation de la masse dans $a = \frac{F}{m}$.

Partie D — Forces de la Nature (Session 4)

D1. Force normale — QCM

La force normale exercée par une surface sur un objet posé dessus :

1. Est toujours **perpendiculaire** à la surface — c'est la surface qui « pousse » l'objet pour l'empêcher de traverser. Sur un plan incliné, elle vaut $mg \cos(\theta)$, ni verticale ni égale au poids.
2. Est toujours **égale au poids** de l'objet — faux sur un plan incliné : $N = mg \cos(\theta) < mg$.
3. Est toujours **verticale**, dirigée vers le haut — faux sur un plan incliné : elle est perpendiculaire au plan, pas verticale.
4. Dépend du **coefficient de friction** de la surface — faux : la normale dépend du poids et de l'angle, pas de μ . C'est la **friction** qui dépend de la normale ($f = \mu N$).

D2. Frottement statique vs cinétique — QCM

Pourquoi est-il plus difficile de **déplacer** un objet immobile que de le **maintenir** en mouvement ?

1. Le frottement **statique** (μ_s) est supérieur au frottement **cinétique** (μ_k) — il faut plus de force pour **décoller** l'objet que pour le garder en mouvement. C'est l'expérience quotidienne : pousser un meuble lourd est difficile au début, puis plus facile une fois qu'il glisse.
2. Le frottement cinétique est supérieur au frottement statique — c'est l'inverse : $\mu_s > \mu_k$.
3. L'objet est plus lourd quand il est immobile — la masse ne change pas avec le mouvement.
4. La force normale augmente avec la vitesse — la normale dépend du poids et de l'angle, pas de la vitesse.

D3. Vitesse terminale — QCM

Qu'est-ce que la **vitesse terminale** ?

1. La vitesse à laquelle la force de **drag** **compense exactement la gravité** — l'accélération devient nulle et l'objet cesse d'accélérer. Il tombe à vitesse **constante**. Pour un drag linéaire : $v_\infty = m \frac{g}{k}$.
2. La vitesse maximale qu'un objet peut atteindre avant de se briser — c'est une limite matérielle, pas physique.
3. La vitesse à laquelle un objet touche le sol — c'est la vitesse d'impact, pas la vitesse terminale.
4. La vitesse initiale qu'il faut donner à un objet pour qu'il tombe — la vitesse terminale est atteinte **naturellement** par le drag, pas donnée initialement.

Partie E — Intégration d'Euler (Session 5)

E1. La chaîne d'Euler — QCM

Dans quel ordre doit-on mettre à jour les quantités à chaque frame avec la méthode d'Euler (semi-implicite) ?

1. Force $\rightarrow$ Accélération $\rightarrow$ Vitesse $\rightarrow$ Position — c'est la **chaîne d'Euler** : on calcule les forces, on dérive l'accélération via $F = ma$, puis on intègre la vitesse, puis la position. $\vec{F} \rightarrow \vec{a} \rightarrow \vec{v} \rightarrow \vec{r}$.
2. Position $\rightarrow$ Vitesse $\rightarrow$ Accélération $\rightarrow$ Force — c'est l'ordre **inverse** ; on ne peut pas calculer la position avant la vitesse.
3. Accélération $\rightarrow$ Force $\rightarrow$ Position $\rightarrow$ Vitesse — la force **cause** l'accélération, pas l'inverse.
4. Vitesse $\rightarrow$ Position $\rightarrow$ Force $\rightarrow$ Accélération — la force doit venir **en premier** ; tout découle d'elle.

E2. Tracer le code — QCM

Voici un extrait de `updatePhysics(dt)` :

```
force.set(0, 0, 0); // ligne 1
force.add(gravity.clone().multiplyScalar(mass)); // ligne 2
acceleration.copy(force).divideScalar(mass); // ligne 3
velocity.addScaledVector(acceleration, dt); // ligne 4
position.addScaledVector(velocity, dt); // ligne 5
```

E2a. Pourquoi remet-on force à zéro (ligne 1) au début de chaque frame ?

1. Pour éviter que les forces s'accumulent d'une frame à l'autre — sans cela, la gravité de la frame n s'ajouterait à celle de la frame $n - 1$, et la force grandirait indéfiniment. La balle s'envolerait.
2. Pour libérer la mémoire du vecteur force — `set(0,0,0)` ne libère rien, il écrase les valeurs.
3. Pour remettre la position à l'origine — force n'a rien à voir avec la position.
4. C'est optionnel, ça ne change rien — sans le reset, les forces s'accumulent et la simulation explose.

E2b. Avec $m = 2$ kg et $\vec{g} = \begin{pmatrix} 0 \\ -9.81 \\ 0 \end{pmatrix}$, que vaut `acceleration` après la ligne 3 ?

1. $\vec{a} = \vec{g} = \begin{pmatrix} 0 \\ -9.81 \\ 0 \end{pmatrix}$ — car $\vec{a} = \frac{\vec{F}}{m} = \frac{m\vec{g}}{m} = \vec{g}$. La masse s'annule : en chute libre, l'accélération est toujours $\vec{g}$, indépendamment de m .
2. $\vec{a} = \begin{pmatrix} 0 \\ -19.62 \\ 0 \end{pmatrix}$ — on a oublié que la masse s'annule. $F = mg = 2 \times 9.81$, mais $a = \frac{F}{m} = \frac{19.62}{2} = 9.81$.
3. $\vec{a} = \begin{pmatrix} 0 \\ -4.905 \\ 0 \end{pmatrix}$ — on a divisé deux fois par la masse.
4. $\vec{a} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$ — la force et la masse ne s'annulent pas ; $a = \frac{F}{m} = g$.

E2c. La ligne 5 utilise la vitesse **déjà mise à jour** (ligne 4) pour intégrer la position. Comment appelle-t-on cette variante ?

1. **Euler semi-implicite** (symplectique) — elle conserve mieux l'énergie que l'Euler classique et est le choix par défaut dans les moteurs de jeu. On utilise la **nouvelle** vitesse pour intégrer la position.

2. **Euler classique** — l'Euler classique utilise l'ancienne vitesse v_n pour la position, pas la nouvelle.
3. **Verlet** — Verlet ne stocke pas la vitesse explicitement, c'est la méthode de la session 8.
4. **Runge-Kutta** — c'est une méthode d'ordre 4, beaucoup plus complexe.

E3. Stabilité d'Euler — QCM

E3a. Pourquoi la méthode d'Euler « dérive » en énergie sur les systèmes conservatifs (ressorts, orbites) ?

1. Euler ignore la **courbure** de la trajectoire (l'accélération) à l'intérieur du pas — l'énergie ajoutée par l'erreur n'est jamais dissipée et s'accumule jusqu'à l'explosion. C'est pourquoi on a banni Euler pour le tissu (session 8).
2. Euler calcule l'accélération trop précisément, ce qui crée des oscillations — c'est le contraire : Euler est **imprécis**.
3. Euler utilise un pas de temps trop grand par défaut — le pas n'est pas le problème, c'est la **méthode** elle-même.
4. Euler ne conserve pas la quantité de mouvement — Euler conserve la quantité de mouvement, c'est l'énergie qui dérive.

E3b. Pourquoi Euler est-il malgré tout acceptable pour une balle qui rebondit avec restitution < 1 ?

1. Chaque rebond **dissipe** de l'énergie — le système est globalement **dissipatif**, pas conservatif. L'erreur d'Euler est invisible car la dissipation est plus forte que la dérive.
2. La balle est trop légère pour que l'erreur soit visible — la masse n'a rien à voir avec la dérive d'Euler.
3. Euler est exact pour les rebonds — Euler n'est jamais exact, il est de premier ordre.
4. La gravité compense exactement l'erreur d'Euler — la gravité n'a aucun lien avec l'erreur de troncature.

E3c. Quel est le rôle du `Math.min(dt, 0.1)` dans la boucle d'animation ?

1. Il **plafonne** le pas de temps — si l'utilisateur change d'onglet, `getDelta()` renvoie un dt énorme (plusieurs secondes) qui ferait exploser la simulation. Le clamp évite ce piège classique.
2. Il accélère la simulation en limitant le dt — le clamp **ralentit** le dt quand il est trop grand, il ne l'accélère pas.
3. Il limite le FPS à 10 images par seconde — le clamp agit sur dt , pas sur le FPS.
4. Il empêche la balle de tomber trop vite — le clamp limite le **pas de temps**, pas la vitesse de la balle.

Partie F — Impulsion et Collisions (Session 6)

F1. Conservation — QCM

Dans une collision entre deux objets (système isolé, pas de forces externes), quelle quantité est **toujours** conservée, élastique ou non ?

1. La **quantité de mouvement totale** ($\vec{p}_A + \vec{p}_B$) — c'est la loi fondamentale des collisions : ce qui est perdu par un objet est gagné par l'autre. Valable pour **toutes** les collisions.
2. L'**énergie cinétique totale** — seulement conservée dans les collisions **élastiques** ($e = 1$). Perdue dans les collisions inélastiques.
3. La **vitesse totale** — les vitesses changent individuellement ; il n'y a pas de « conservation de vitesse ».
4. La **masse totale** — oui, mais c'est trivial. La quantité de mouvement est la quantité physique **pertinente**.

F2. Restitution — QCM

Le coefficient de restitution e est défini par :

$$e = -\frac{v_{\text{relative après}}}{v_{\text{relative avant}}}$$

Si $e = 0.5$ et la vitesse relative avant l'impact est 10 m/s, que vaut la vitesse relative après l'impact ?

1. -5 m/s — la formule donne $v_{\text{rel après}} = -e \times v_{\text{rel avant}} = -0.5 \times 10 = -5$. Le signe négatif signifie que les objets se **séparent** (sens opposé), et la magnitude est divisée par 2.
2. 10 m/s — cela correspondrait à $e = 1$ (élastique parfait), pas $e = 0.5$.
3. 5 m/s (même sens) — le signe est faux. Après l'impact, les objets se **séparent**, le sens s'inverse.
4. 0 m/s — cela correspondrait à $e = 0$ (les objets collent), pas $e = 0.5$.

F3. Impulsion — QCM

Dans le code de résolution de collision, on trouve cette garde :

```
const vRel = relVel.dot(normal);  
if (vRel > 0) return; // <-- on arrête tout
```

F3a. Que signifie $v_{\text{rel}} > 0$?

1. Les deux objets **s'éloignent** déjà l'un de l'autre — la composante normale de leur vitesse relative est positive, ils se séparent. Il n'y a pas de collision à résoudre.
2. Les deux objets **se rapprochent** — s'ils se rapprochent, $v_{\text{rel}} < 0$ (négatif), pas positif.
3. Les deux objets sont **immobiles** — si $v_{\text{rel}} = 0$, ils sont immobiles relativement, pas si $v_{\text{rel}} > 0$.
4. La collision est **élastique** — v_{rel} ne dit rien sur le type de collision, seulement sur la direction.

F3b. Que se passerait-il si on retirait cette garde (`if (vRel > 0) return;`) ?

1. On appliquerait une impulsion à des objets qui **se séparent** — ils s'éloigneraient encore plus. C'est une impulsion « fantôme » qui **ajoute** de l'énergie au lieu de résoudre la collision.
2. Rien — la garde est optionnelle — sans la garde, le code applique l'impulsion même quand elle ne devrait pas.
3. Les objets passeraient à **travers** le sol — la garde n'a rien à voir avec le sol, elle gère les collisions entre objets.

4. La simulation s'arrêterait — la simulation continue, mais avec des impulsions incorrectes.

Partie G — Détection de Collision : Broad et Narrow Phase (Session 7)

G1. Complexité de la brute force — QCM

Avec $N = 1000$ objets, combien de paires la brute force génère-t-elle ?

1. 499500 — la formule est $\frac{N(N-1)}{2} = 1000 \times \frac{999}{2}$, car chaque paire n'est testée qu'une fois. C'est le $O(N^2)$ qui rend la brute force inutilisable au-delà de quelques dizaines d'objets.
2. 1000000 — c'est N^2 , mais ça compte chaque paire **deux fois** (A-B et B-A). La vraie formule divise par 2.
3. 1000 — une seule paire par objet, mais chaque objet doit tester **tous** les autres.
4. 999000 — c'est $N(N - 1)$, mais ça double les paires. On divise par 2 pour éviter les doublons.

G2. AABB — QCM

Pourquoi utilise-t-on des boîtes **alignées sur les axes** (AABB) pour la Broad Phase ?

1. Parce que le test de chevauchement est **trivial** — quelques comparaisons par axe, pas de trigonométrie ni de produit vectoriel. C'est le test le plus rapide qui existe en 3D. Les objets n'ont pas besoin d'être alignés ; c'est la **boîte englobante** qui l'est.
2. Parce que les objets sont toujours alignés sur les axes dans un jeu — les objets peuvent tourner, mais leur **boîte englobante** reste alignée pour la performance.
3. Parce que c'est la seule façon de calculer une boîte englobante — on peut aussi utiliser des OBB (Oriented Bounding Box), mais le test est plus coûteux.
4. Parce que la rotation des objets n'a pas d'importance en physique — la rotation importe en Narrow Phase, mais la Broad Phase la ignore volontairement pour la performance.

G3. Structures de partitionnement — QCM

Associez chaque structure à son cas d'usage idéal :

1. **Grille uniforme** →
2. **Quadtree/Octree** →
3. **Sweep and Prune** →

Options (une par structure) : a. **Monde ouvert**, zones denses et vides, culling de frustum — l'arbre s'adapte à la densité en subdivisant les zones denses. b. **Particules de taille similaire** dans une arène fermée — accès $O(1)$, très rapide, facile à coder. c. **Objets de tailles hétérogènes**, cohérence temporelle exploitée — tri presque gratuit d'une frame à l'autre car les objets bougent peu.

G4. Budget temps — QCM

À 60 FPS, on dispose de 16.6 ms par frame. Pourquoi la Broad Phase est-elle le **goulot d'étranglement numéro un** des moteurs physiques ?

1. Parce qu'elle **scale avec N** — c'est elle qui détermine combien de paires sont testées. Si elle est lente, tout le reste cascade (Narrow Phase, Response). Elle doit consommer **moins de 0.5 ms** sur les 2 ms budget physique.
2. Parce que c'est l'étape la plus précise géométriquement — c'est la **moins** précise (AABB simplifiées). La précision est en Narrow Phase.

3. Parce qu'elle calcule les impulsions de collision — c'est la **Response** qui calcule les impulsions, pas la Broad Phase.
4. Parce qu'elle est la dernière étape du pipeline — c'est la **première** étape. C'est justement parce qu'elle est première qu'elle est le goulot.

Partie H — Verlet et PBD (Session 8)

H1. Verlet vs Euler — QCM

Quelle est la principale différence entre Verlet et Euler ?

1. Verlet **déduit** la vitesse de $x_n - x_{n-1}$ (vitesse implicite), Euler **stocke** v explicitement — c'est la différence fondamentale. Verlet n'a pas de variable velocity ; la vitesse émerge de la différence des positions.
2. Verlet **stocke** la vitesse explicitement, Euler non — c'est l'inverse : Euler stocke v , Verlet la déduit.
3. Verlet est du **premier ordre**, Euler du deuxième — c'est l'inverse : Verlet est $O(dt^4)$, Euler $O(dt^3)$.
4. Il n'y a pas de différence, c'est la même méthode — les deux méthodes ont des propriétés très différentes (stabilité, précision, gestion des contraintes).

H2. Verlet et PBD — QCM

H2a. Pourquoi Verlet est-elle **plus précise** qu'Euler ?

1. Verlet est $O(dt^4)$ tandis qu'Euler est $O(dt^3)$ — la dérivation par Taylor montre qu'en additionnant $x(t + dt)$ et $x(t - dt)$, les termes en v_n s'annulent, laissant une erreur d'ordre supérieur.
2. Verlet est $O(dt^3)$ comme Euler — faux, Verlet est d'un ordre supérieur.
3. Verlet utilise des pas de temps plus petits — le pas de temps est le même, c'est la **méthode** qui est plus précise.
4. Verlet ne utilise pas l'accélération — Verlet utilise bien $a \cdot dt^2$ dans sa formule.

H2b. Pourquoi PBD **exige** Verlet (et ne fonctionne pas bien avec Euler) ?

1. PBD **déplace directement** les positions pour satisfaire les contraintes. Avec Verlet, la vitesse étant $x_n - x_{n-1}$, **toute correction de position met à jour automatiquement la vitesse**. Avec Euler, le déplacement manuel n'affecte pas v — la vitesse devient incohérente et le système explose.
2. Parce qu'Euler est moins précis — la précision n'est pas le problème ici, c'est la **cohérence** position/vitesse.
3. Parce que Verlet est plus rapide — la vitesse n'est pas la question, c'est la **gestion des contraintes**.
4. Parce qu'Euler ne supporte pas les forces — Euler supporte parfaitement les forces, c'est le déplacement **manuel** qui pose problème.

H3. Contrainte PBD — QCM

Voici la fonction satisfyConstraint du TP :

```
function satisfyConstraint(p1, p2, restLength) {  
  const delta = p2.position.clone().sub(p1.position);  
  const dist = delta.length();  
  if (dist === 0) return;  
  const diff = (dist - restLength) / dist;  
  const correction = delta.multiplyScalar(0.5 * diff);  
  if (!p1.pinned) p1.position.add(correction);  
  if (!p2.pinned) p2.position.sub(correction);  
}
```

H3a. Que représente $\text{diff} = (\text{dist} - \text{restLength}) / \text{dist}$?

1. L'**écart relatif** — on divise par dist pour normaliser. La correction sera proportionnelle à delta (la direction), pas à la distance absolue. C'est un pourcentage d'étirement.
2. L'**écart absolu** — l'écart absolu est $\text{dist} - \text{restLength}$, sans la division.
3. La distance au repos — c'est restLength , pas diff .
4. La direction de la correction — la direction est delta , pas diff .

H3b. Pourquoi corrige-t-on de $0.5 * \text{diff}$ et non diff ?

1. Parce qu'on répartit l'écart **équitablement** entre les deux particules — chacune absorbe la **moitié** de la correction. Si on corrigeait de diff , chaque particule se déplacerait de l'écart **total**, doublant la correction.
2. Parce que 0.5 est un coefficient de damping — le damping est une autre chose (section 3 du cours).
3. Parce que la contrainte est **semi-rigide** — la rigidité vient des **itérations**, pas du facteur 0.5.
4. C'est un choix arbitraire — non, c'est la **moitié** de l'écart pour chaque particule, ce qui donne la correction **totale** exacte.

H3c. Pourquoi itère-t-on cette fonction N fois par frame ? Que se passe-t-il avec $N = 1$ vs $N = 20$?

1. Corriger une contrainte **déplace ses voisines** — il faut itérer pour propager les corrections jusqu'à convergence. $N = 1$ donne un tissu **élastique** (s'étire), $N = 20$ donne un tissu **rigide**. Les itérations **sont** la raideur en PBD.
2. Plus d'itérations = plus de précision numérique — ce n'est pas une question de précision, c'est la **convergence** d'un système couplé.
3. La formule l'exige mathématiquement — une seule passe suffit mathématiquement pour une contrainte isolée, mais les contraintes sont **couplées**.
4. Pour éviter la division par zéro — la garde `if (dist == 0)` gère déjà ce cas, les itérations n'ont rien à voir.

H4. Damping — QCM

En Verlet, le damping est appliqué comme un coefficient $d \leq 1$ sur la vitesse implicite. Si $d = 0.99$:

1. La particule perd **1%** de sa vitesse par frame — elle conserve 99% de sa vitesse. C'est une **dissipation douce**, typique pour un drapeau qui flotte.
2. La particule perd **99%** de sa vitesse par frame — c'est l'inverse : $d = 0.99$ signifie qu'on **garde** 99%, pas qu'on perd 99%.
3. La particule ne perd **aucune** énergie — c'est seulement si $d = 1$ (aucun amortissement).
4. La particule s'arrête en **une frame** — c'est seulement si $d = 0$ (plus d'inertie).

Partie I – Moteur de Particules (Session 9)

I1. Object Pool – QCM

Pourquoi utilise-t-on un **object pool** plutôt que de créer/détruire des particules à chaque frame ?

1. Pour éviter les pics de **garbage collection** — créer/détruire des objets chaque frame provoque des pauses du GC et des saccades. Un pool alloue **une fois** un tableau fixe et **recycle** les particules mortes. Zéro allocation pendant le jeu.
2. Pour **sauver de la mémoire** — un pool utilise **autant** de mémoire qu'un système dynamique, mais de façon **fixe** et prévisible.
3. Pour rendre les particules **plus rapides** à calculer — la physique des particules est la même ; c'est l'**allocation** qui est évitée.
4. Parce que JavaScript **exige** un object pool — c'est une **optimisation**, pas une obligation du langage.

I2. Euler vs Verlet pour les particules – QCM

Pourquoi l'intégration d'Euler est-elle **légitime** pour un moteur de particules VFX, alors qu'elle était bannie pour le tissu (session 8) ?

1. Les particules vivent **peu de temps**, sont très **amorties** (drag fort), et n'ont pas de **contraintes couplées** — la dérive d'Euler est invisible sur 1 s, le drag dissipe plus que ce qu'Euler ajoute, et il n'y a pas de contraintes de distance à maintenir.
2. Les particules sont **plus légères**, donc Euler est plus précis — la masse n'affecte pas la précision de l'intégrateur.
3. Euler est **toujours** meilleur que Verlet — Euler a été banni pour le tissu précisément parce qu'il dérive.
4. On utilise **Verlet** pour les particules, pas Euler — le cours dit explicitement qu'on **revient** à Euler pour la VFX.

I3. Forces du feu – QCM

Pour produire un effet de **feu** (les particules **montent**), que doit valoir la « gravité effective »

$$g_{\text{eff}} = g \cdot \left(1 - \frac{\rho_{\text{air}}}{\rho_{\text{particule}}}\right) ?$$

1. **Positive** (vers le haut) — le feu est moins dense que l'air ($\rho_{\text{particule}} < \rho_{\text{air}}$), donc la flottabilité (Archimède) **dépasse** la gravité. Le preset fire utilise gravity: +2.5 : la particule **monte**.
2. **Négative** (vers le bas) — cela produirait de la **pluie**, pas du feu. Les particules tomberaient.
3. **Zéro** — les particules flotteraient immobiles, sans monter ni descendre.
4. Ça dépend de la **taille** des particules — la flottabilité dépend de la **densité**, pas de la taille.

I4. Collision avec le sol – QCM

Quand une particule touche le sol ($y < y_{\text{sol}}$), que doit-on faire à sa vitesse verticale v_y ?

1. **Inverser et multiplier par la restitution** : $v_{y'} = -e \cdot v_y$ — cela donne un rebond amorti. Seule la composante **normale** (y) est affectée ; les composantes tangentielles (v_x, v_z) sont **conservées** (pas de friction de surface).
2. **Mettre à zéro** — la particule s'arrêterait net, sans rebond. C'est le cas $e = 0$.

3. **Inverser sans amortissement** : $v_{y'} = -v_y$ — c'est le cas $e = 1$ (élastique parfait), la particule rebondit indéfiniment.
4. **Multiplier par e sans inverser** : $v_{y'} = e \cdot v_y$ — la particule **accélérerait** vers le bas, ce qui n'a aucun sens.

Partie J – Rotations et Corps Rigides (Session 10)

J1. Analogie linéaire → angulaire – QCM

Associez chaque concept linéaire à son équivalent angulaire :

1. Force $\vec{F} \rightarrow$
2. Masse $m \rightarrow$
3. Vitesse $\vec{v} \rightarrow$
4. Quantité de mouvement $\vec{p} = m\vec{v} \rightarrow$

Options : a. Moment d'inertie I – résistance à la rotation (comme m résiste à la translation). b. Couple $\vec{\tau}$ – cause la rotation (comme $\vec{F}$ cause la translation). c. Vitesse angulaire $\vec{\omega}$ – taux de rotation (comme $\vec{v}$ est le taux de déplacement). d. Moment angulaire $\vec{L} = I\vec{\omega}$ – quantité de mouvement de rotation.

J2. Couple – QCM

Vous poussez une porte. Que détermine le **couple** (et donc la rotation de la porte) ?

1. La force **ET** le bras de levier ($\tau = r \times F$) – pousser **au bout** de la porte (r grand) produit beaucoup de rotation ; pousser **près de la charnière** (r petit) produit peu de rotation ; pousser **sur la charnière** ($r = 0$) ne produit **aucune** rotation, peu importe la force.
2. **Uniquement la force** – la même force produit des couples très différents selon le point d'application.
3. **Uniquement le bras de levier** – sans force, il n'y a pas de couple.
4. La **masse** de la porte – la masse détermine l'inertie (I), pas le couple (τ).

J3. Moment d'inertie – QCM

Une sphère **pleine** a $I = \frac{2}{5}mR^2$. Une sphère **creuse** (même m , même R) a $I = \frac{2}{3}mR^2$. Laquelle est plus difficile à faire tourner ?

1. La sphère **creuse** – sa masse est concentrée **sur le bord** (plus loin de l'axe), donc I est plus grand. Plus la masse est loin de l'axe, plus I est grand, plus il est difficile de faire tourner. (Intuition : le patineur qui écarte les bras tourne plus lentement.)
2. La sphère **pleine** – sa masse est répartie **partout**, donc plus proche de l'axe en moyenne. I est **plus petit**.
3. Elles sont **identiques** – même m et même R , mais la **répartition** de la masse diffère, et c'est ça qui compte.
4. Ça dépend de la **vitesse** – le moment d'inertie est une propriété **géométrique** de l'objet, indépendante de la vitesse.

J4. Roulement sans glissement – QCM

J4a. Quelle est la condition de roulement sans glissement ?

1. $v_G = \omega R$ – la vitesse du centre égale la vitesse tangentielle de rotation. Le point de contact est **immobile** par rapport au sol ($\vec{v}_P = 0$).
2. $v_G = 0$ – la boule ne bouge pas, ce n'est pas du roulement.
3. $\omega = 0$ – la boule ne tourne pas, c'est du glissement pur.
4. $v_G = \omega$ – il manque le rayon R ; les unités ne correspondent pas (m/s vs rad/s).

J4b. Pendant la phase de glissement d'une boule de billard, que fait la friction cinétique ?

1. Elle **ralentit** la translation (v_G diminue) ET **accélère** la rotation (ω augmente) — la même force de friction fait les deux : elle décélère le centre de masse tout en créant un couple qui accélère la rotation. Les deux convergent vers $v_G = \omega R$.
2. Elle **ralentit** uniquement la translation — elle crée aussi un couple, donc elle accélère aussi la rotation.
3. Elle **accélère** uniquement la rotation — elle décélère aussi la translation via $\vec{a} = \frac{\vec{F}}{m}$.
4. Elle **accélère** les deux — la friction **s'oppose** au glissement, elle ne l'accélère pas.

J5. Quaternions — QCM

Pourquoi utilise-t-on des quaternions plutôt que des angles d'Euler en 3D ?

1. Les angles d'Euler souffrent du **gimbal lock** (perte d'un degré de liberté quand deux axes s'alignent) et ne s'interpolent pas **naturellement** (le chemin le plus court n'est pas pris) — les quaternions résolvent ces deux problèmes.
2. Les quaternions sont **plus rapides** à calculer — la vitesse n'est pas la raison principale, c'est la **qualité** de la rotation.
3. Les quaternions sont **plus intuitifs** pour les artistes — c'est l'inverse : les angles d'Euler sont plus intuitifs mais bugués.
4. Les angles d'Euler ne peuvent **pas** représenter toutes les rotations — ils le peuvent, mais avec des problèmes (gimbal lock, interpolation).

J6. Intégration de la rotation — QCM

```
const axis = angularVelocity.clone().normalize();
const angle = angularVelocity.length() * dt;
if (angle > 1e-6) {
    const dq = new THREE.Quaternion().setFromAxisAngle(axis, angle);
    quaternion.premultiply(dq);
}
```

J6a. Pourquoi utilise-t-on `premultiply` plutôt que `multiply` ?

1. `premultiply` applique la rotation dans le **repère monde** ($q_{\text{new}} = dq \cdot q_{\text{old}}$) — pour un corps rigide, on veut généralement la rotation dans le repère monde. `multiply` l'appliquerait dans le **repère local** de l'objet.
2. `premultiply` est **plus rapide** — la vitesse n'est pas la raison, c'est le **repère** de la rotation.
3. `premultiply` **évite le gimbal lock** — le gimbal lock est un problème des angles d'Euler, pas des quaternions.
4. Il n'y a **aucune différence** — `premultiply` et `multiply` donnent des résultats **différents** (repère monde vs local).

J6b. À quoi sert la garde `if (angle > 1e-6)` ?

1. À éviter de créer un quaternion depuis un vecteur **nul** — si `angularVelocity` est nul, `normalize()` produit un vecteur invalide (division par zéro, NaN). La garde saute la mise à jour quand la rotation est négligeable.
2. À **optimiser** les performances — c'est une **correction** de robustesse, pas une optimisation.
3. À empêcher le quaternion de **grandir** indéfiniment — les quaternions sont normalisés, ils ne grandissent pas.
4. À **limiter** la vitesse de rotation — la garde ne limite rien, elle saute juste le cas trivial.

Partie K — Synthèse

K1. Choix de la méthode d'intégration — QCM

Associez chaque scénario à la meilleure méthode d'intégration :

1. a) Pluie : 3000 gouttes éphémères () →
2. b) Drapeau qui flotte pendant des heures →
3. c) Boule de billard qui s'arrête en quelques secondes →
4. d) Corde suspendue avec interaction du joueur →

Options (une par scénario) : a. **Euler** — la boule est très dissipative (friction), la dérive est invisible sur quelques secondes. b. **Verlet + PBD** — le drapeau oscille pendant des heures, Euler dériverait en énergie. PBD maintient les contraintes sans calcul de forces. c. **Euler** — les gouttes vivent quelques secondes, sont très amorties (drag), pas de contraintes couplées. La dérive est invisible. d. **Verlet + PBD** — une corde est un système de contraintes de distance couplées. Verlet tolère les déplacements manuels du joueur.

K2. Le pipeline de collision — QCM

Pourquoi le pipeline de collision utilise-t-il un **entonnoir** (Broad → Narrow → Response) plutôt que de tester toutes les paires ?

1. Parce que la brute force est en $O(N^2)$ — inutilisable au-delà de quelques dizaines d'objets.
L'entonnoir **élimine** des candidats à chaque étage (95–99% à la Broad Phase), divisant le travail par un ordre de grandeur à chaque fois. Chaque étage est **moins cher** que le suivant.
2. Parce que c'est **plus précis** géométriquement — c'est l'inverse : chaque étage est **moins** précis mais **plus** rapide que le suivant.
3. Parce que c'est **requis** par la physique — l'entonnoir est une **optimisation** d'ingénierie, pas une nécessité physique.
4. Parce que ça utilise **moins de mémoire** — l'entonnoir vise la **vitesse**, pas la mémoire.

K3. Debug — QCM

Un étudiant a écrit ce code pour un drapeau Verlet + PBD. Le drapeau « explose ». Identifiez les bugs :

```
function verletIntegrate(p, dt) {
    const velocity = p.position.clone().sub(p.prevPosition);
    p.position.add(velocity);
    p.position.addScaledVector(p.acceleration, dt); // ligne A
    p.prevPosition.copy(p.position);                // ligne B
}

function satisfyConstraint(p1, p2, restLength) {
    const delta = p1.position.clone().sub(p2.position);
    const dist = delta.length();
    const diff = (dist - restLength) / restLength; // ligne C
    const correction = delta.multiplyScalar(diff);
    p1.position.add(correction);
    p2.position.add(correction);                    // ligne D
}
```

K3a. Bug sur la ligne A : `addScaledVector(p.acceleration, dt)`

1. Devrait être $dt * dt$ — en Verlet, le terme d'accélération est $a \cdot dt^2$, pas $a \cdot dt$. Avec dt au lieu de dt^2 , l'accélération est 60 fois trop forte (à 60 FPS), ce qui fait exploser les particules.
2. Devrait être $dt / 2$ — il n'y a pas de facteur $\frac{1}{2}$ dans le terme d'accélération de Verlet.
3. Le code est correct — non, Verlet utilise dt^2 , pas dt .
4. Il faut **retirer** la ligne — l'accélération est nécessaire (gravité, vent).

K3b. Bug sur la ligne B : `p.prevPosition.copy(p.position)`

1. Il faut sauvegarder l'ancienne position **avant** de la modifier — on copie la **nouvelle** position dans `prevPosition`, mais elle a déjà été modifiée. La vitesse implicite ($x_n - x_{n-1}$) devient nulle ou incohérente. Il faut cloner `temp` au début, puis `p.prevPosition.copy(temp)` à la fin.
2. Le code est correct — non, on écrase `prevPosition` avec la position **déjà modifiée**.
3. Il faut retirer la ligne — sans cette ligne, la vitesse implicite ne se met pas à jour.
4. Il faut copier **après** la fonction — la copie doit se faire **à la fin** de la fonction, mais sur l'ancienne position sauvegardée.

K3c. Bug sur la ligne D : `p2.position.add(correction)`

1. Devrait être `p2.position.sub(correction)` — les deux particules doivent être corrigées en **sens opposé** (l'une rapprochée, l'autre éloignée). Avec `add` pour les deux, elles se déplacent dans la **même** direction — la contrainte n'est jamais satisfaite et le système diverge.
2. Le code est correct — non, les deux particules se déplacent dans le **même** sens, ce qui est physiquement faux.
3. Devrait être `p2.position.add(correction.clone().multiplyScalar(2))` — cela aggrave le problème.
4. Il faut retirer la ligne — sans corriger `p2`, seule une particule est déplacée.